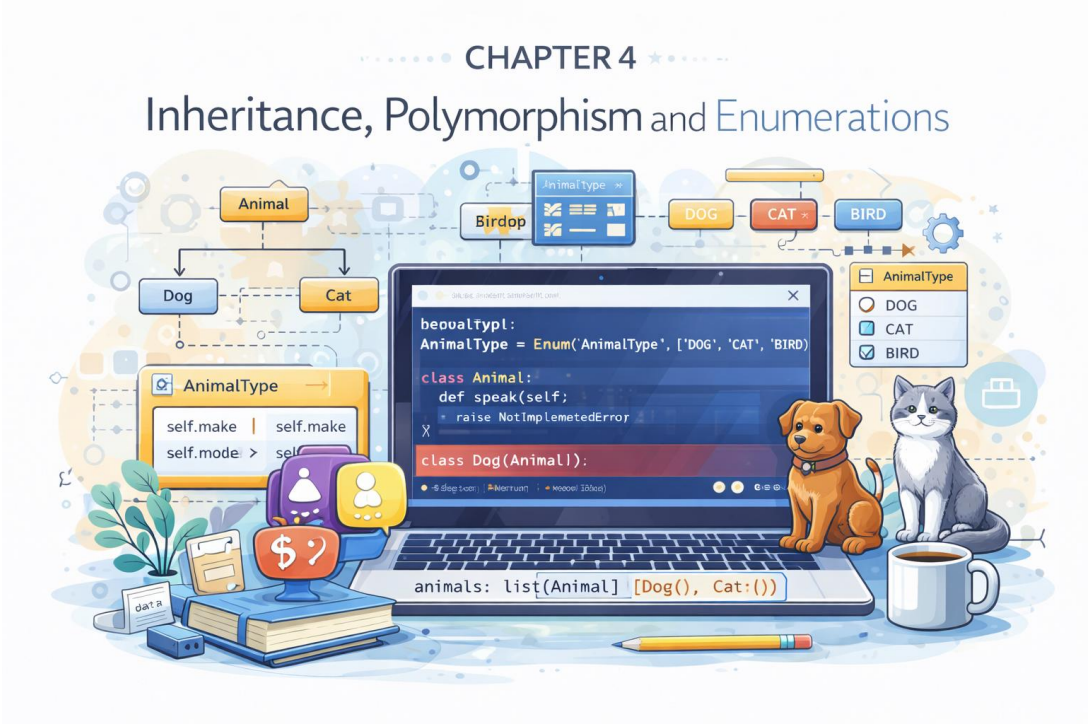

The following was generated using ChatGPT and further edited by Lucas Hartman

Chapter 4 – Inheritance, Polymorphism and Enumerations

Programming in Python often involves recognizing real-world patterns and relationships, then modeling them through code. As systems grow, many classes share similar traits or methods. Instead of rewriting code repeatedly, Python allows you to reuse and extend existing classes through inheritance. Inheritance defines logical relationships between objects — for example, a Car is a type of Vehicle, or a Manager is a type of Employee. By inheriting shared functionality, subclasses can focus on what makes them unique while still benefiting from the base class structure.

Python also supports polymorphism, meaning “many forms,” which lets different classes respond to the same method in ways appropriate to their type. For instance, both a Dog and a Cat may define a speak() method, but each produces its own sound. This allows you to write flexible, general-purpose code that can handle diverse object types without knowing their specific implementations. Polymorphism simplifies programs, improves readability, and helps systems grow without becoming tangled in conditional logic.

A related feature that enhances organization is the use of enumerations (enums) — special classes that represent a fixed set of constant values such as employee roles, access levels, or departments. Enums improve clarity by grouping meaningful names instead of relying on arbitrary strings or numbers, while still benefiting from Python’s class and inheritance model.

4.1 Inheritance Concepts

Inheritance allows one class (the subclass) to reuse and extend another class (the parent or base class). It models an 'is-a' relationship—for example, a Dog is an Animal. Through inheritance, you can define shared data and behavior once in the parent class, then specialize it in subclasses.

To create inheritance when defining the class, you put a set of parenthesis that note the base class.

```
class Animal:
    def __init__(self, name):
        self._name = name

    def speak(self):
        return "<silence>"

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

animals = [Dog("Rex"), Cat("Mittens")]
for a in animals:
    print(a.speak())
```

Here, the Dog and Cat subclasses both inherit the constructor and structure from Animal but override the speak() method to customize behavior. This lets us reuse the Animal structure while adapting it to different needs.

4.1.1 Access Levels (Revisited)

When working with inheritance, it's important to understand how different types of variables behave between parent (super) and child(sub) classes. Python doesn't have strict access control as we discussed in the last chapter but we still need to use the naming conventions we covered.

- public variables – no underscores and directly accessible from outside the class.
- private variables – two underscores and only directly accessible from inside the class. Use getter/setter methods to access variables or change their values from outside the class.
- protected variables – one underscore and only directly accessibly from within the class or it's sub classes. Use getter/setter methods to access variable or change values from outside the class.

This helps us understand the agreement we lay out in the code. If we have values that need to be accessible in sub classes, make sure they are protected. If we don't want to share access then keep them private and use the getter/setters.

4.1.2 Using super()

When a subclass overrides the constructor (__init__) or another method, you can still call the parent's version using the built-in function super(). This ensures that the parent's setup code runs correctly and allows you to extend behavior instead of completely replacing it.

```
class Vehicle:
    def __init__(self, make):
        self._make = make

class Car(Vehicle):
    def __init__(self, make, model):
        super().__init__(make)
        self._model = model

    def info(self):
        return f"{self._make} {self._model}"

car = Car("Toyota", "Camry")
print(car.info()) # Toyota Camry
```

The super() call ensures the Vehicle constructor runs first to set the make before Car adds its own model attribute. Using super() prevents duplicate code and maintains consistent initialization across the class hierarchy.

You can also user the super() to call methods from the base class. We will see this coming up with overriding methods and adding to them.

4.1.3 Using isinstance() and issubclass()

Python provides two helpful functions for checking object relationships:

- isinstance(obj, Class) returns True if obj is an instance of Class or any subclass of it.
- issubclass(Sub, Base) returns True if Sub inherits from Base.

```
print(isinstance(Circle(3), Circle))       # True
print(isinstance(Rectangle(4,5), Circle))  # False
print(issubclass(Rectangle, object))       # True
```

While these functions are useful for validation and debugging, try to rely on polymorphism whenever possible rather than manually checking types.

4.2 Polymorphism

Polymorphism means 'many forms'—it allows different objects to respond to the same method in their own way. In Python, polymorphism happens naturally since you don't need to declare types in advance. As long as an object defines the method you're calling, it can participate in polymorphic behavior.

```
class Shape:
    def area(self):
        raise NotImplementedError("Subclasses must implement area")

class Circle(Shape):
    def __init__(self, radius):
        self._radius = radius

    def area(self):
        return 3.14159 * self._radius ** 2

    def __str__(self):
        return f"Circle with radius {self._radius}, Area: {self.area():.2f}"

class Rectangle(Shape):
    def __init__(self, width, height):
        self._width = width
        self._height = height

    def area(self):
        return self._width * self._height

    def __str__(self):
        return f"Rectangle {self._width}x{self._height}, Area: {self.area():.2f}"

shapes = [Circle(3), Rectangle(4, 5), Circle(6)]
for s in shapes:
    print(s)
```

The loop treats all shapes the same way. Each object has an area() method, but their behavior depends on which class the object belongs to. This is polymorphism—one interface, many implementations.

Here we also implement an exception being thrown to force an area() method to be stated. So you must override it in the sub classes.

4.4 Single and Multiple Inheritance

Python supports both single and multiple inheritance. In single inheritance, a class inherits from one parent. In multiple inheritance, it can inherit from several parents. While multiple inheritance is powerful, it can also make code harder to understand.

```
class JSONSerializable:
    def to_json(self):
        import json
        return json.dumps(self.__dict__)

class Printable:
    def __str__(self):
        return f"<{self.__class__.__name__}: self.__dict__}>"

class User(JSONSerializable, Printable):
    def __init__(self, name):
        self._name = name

u = User("Alice")
print(u)
print(u.to_json())
```

Here, the User class inherits from both JSONSerializable and Printable. This pattern is often used for mixins—small, reusable classes that add specific features like logging or formatting. Python resolves method calls left to right using the Method Resolution Order (MRO).

4.6 Advantages and Disadvantages of Inheritance

Advantages:

- **Code reuse**: Share common logic in a single base class.
- **Specialization**: Subclasses refine or extend behavior without duplicating code.
- **Polymorphism**: Call the same method on different objects and get appropriate behavior.

Disadvantages:

- **Tight coupling**: Subclasses depend on base class internals. Changes ripple outward.
- **Fragile hierarchies**: Deep or complicated inheritance trees are hard to understand and maintain.
- **Wrong relationship**: Sometimes you choose inheritance when you actually need composition ("has-a").
  Example: A Car has an Engine—that's not inheritance.

Guideline:

- Use inheritance for clear is-a relationships and shared behavior.
- Use composition (embed objects) for has-a relationships or when behavior is better delegated than inherited.
- For cross-cutting features (e.g., logging, serialization), consider mixins or helper objects.
  - Mixins are often small classes that end with the word "Mixin" that are not standalone classes. Ideally they have no state and should work purely by using attributes that already exist in the class they are mixed into.
  - Designed to be safely stacked together in a class
  - Think of these as a utility class

Diamond Problem issues

Python resolves methods using MRO (Method Resolution Order): left-to-right across the inheritance list, then up the hierarchy.

```
  A
 / \
B   C
 \ /
  D
```

If B and C both override a method from A, and C inherits from B and C, which version does D use? Python's MRO resolves this deterministically, but designs like this can become hard to read and maintain. For this we prefer composition or mixins when things get complex.

4.7 Example: Employee Payroll System

This example shows how inheritance and polymorphism can model an employee system where each subclass calculates pay differently.

```
class Employee:
    def __init__(self, name : str, department : str):
        self._name = name
        self._department = department

    def __str__(self):
        return f"Employee Name: {self._name}\nDepartment : {self._department}"

class SalariedEmployee(Employee):
    def __init__(self, name : str, dept : str, annual_salary : float):
        super().__init__(name, dept)
        self._annual_salary = annual_salary

    def weekly_pay(self):
        return self._annual_salary / 52

    def __str__(self):
        return f"{super().__str__()}\nWeekly Pay: ${self.weekly_pay():.2f}"

class HourlyEmployee(Employee):
    def __init__(self, name : str, dept : str, hourly_rate : float, hours_worked : float):
        super().__init__(name, dept)
        self._hourly_rate = hourly_rate
        self._hours_worked = hours_worked

    def weekly_pay(self):
        return self._hourly_rate * self._hours_worked

    def __str__(self):
        return f"{super().__str__()}\nWeekly Pay: ${self.weekly_pay():.2f}"


employees = [
    SalariedEmployee("Alice", "Manager", 96000),
    HourlyEmployee("Bob", "Sales Support", 18.75, 36)
]

for e in employees:
    print(e)
```

4.8 Enumerations in Python

When programs grow larger, you often need to represent a fixed set of related constant values—such as days of the week, access levels, or job roles. Python's enum module provides a clear, type-safe way to group these constants while keeping your code readable and consistent. Enums also demonstrate inheritance in action since each enumeration class extends the built-in Enum base class.

4.8.1 What Are Enums?

An enumeration (Enum) is a special class that defines a group of symbolic names bound to constant values. Each member of an Enum is unique and immutable, which prevents accidental re-assignment or confusion between similar constants. Enums are especially helpful for improving code readability and preventing errors caused by mistyped strings or inconsistent numeric values.

```
role = "manager"
if role == "Manager":  # Typo, won't work
    print("Access granted.")
```

4.8.2 Enum referencing Integers:

```
from enum import Enum

class Role(Enum):
    MANAGER = 1
    SALES = 2
    SUPPORT = 3

role = Role.MANAGER
if role == Role.MANAGER:
    print("Access granted.")
```

Each member has both a name and a value. Using Enums makes code easier to read and helps prevent logic errors that occur when using raw strings or integers.

4.8.3 Enum referencing string:

```
from enum import Enum

class Department(Enum):
    ENGINEERING = "Engineering"
    SALES = "Sales"
    SUPPORT = "Customer Support"

for dept in Department:
    print(dept.name, "=>", dept.value)

print(Department.ENGINEERING.name)
print(Department.ENGINEERING.value)
```

Here, each Department member has a string value. Enums can store strings, numbers, or any immutable type. Iterating through an Enum yields all its defined members in order.

4.8.4 Enum Types: Enum vs IntEnum vs StrEnum

Python's enum module provides different base classes that extend Enum to support specific data behaviors. Choosing the right base class determines whether members act like plain objects, integers, or strings.

```
from enum import Enum, IntEnum, StrEnum

# Basic Enum (strict identity comparison)
class Status(Enum):
    PENDING = 1
    APPROVED = 2
    REJECTED = 3

print(Status.PENDING == 1)  # False

# IntEnum (acts like an integer)
class AccessLevel(IntEnum):
    GUEST = 0
    USER = 1
    ADMIN = 2

print(AccessLevel.ADMIN == 2)  # True
print(AccessLevel.ADMIN > AccessLevel.USER)  # True

# StrEnum (acts like a string, Python 3.11+)
class State(StrEnum):
    OPEN = "Open"
    CLOSED = "Closed"

print(State.OPEN == "Open")  # True
print(State.CLOSED.lower())  # Works like a string
```

Enum members are unique objects, while IntEnum and StrEnum behave like their respective base types. This means you can perform numeric or string operations with them directly while maintaining the clarity of named constants.

- Use Enum for symbolic constants when you want strict type comparison.
- Use IntEnum when values should behave like integers.
- Use StrEnum when members should behave like strings (Python 3.11+).

4.8.5 Enums Inside Classes

Enums can also be integrated into larger object-oriented systems. They are often used inside classes to represent categories, roles, or states safely and consistently.

```
from enum import Enum

class Role(Enum):
    MANAGER = "Manager"
    DEVELOPER = "Developer"
    INTERN = "Intern"

class Employee:
    def __init__(self, name: str, role: Role):
        self._name = name
        self._role = role

    def __str__(self):
        return f"Employee: {self._name}, Role: {self._role.value}"

emp1 = Employee("Alice", Role.MANAGER)
emp2 = Employee("Bob", Role.DEVELOPER)

print(emp1)
print(emp2)
```

4.8.6 Looping Through Enums and Displaying Choices

While enums are often used inside objects to represent roles or categories, they are also incredibly useful for displaying options to a user. Because an Enum behaves like a small iterable collection, you can loop through its members to dynamically build menus, dropdowns, or selection lists. This approach ensures that user interface options always stay consistent with the program's logic.

Example – Displaying Choices from an Enum

```
from enum import Enum

class Department(Enum):
    ENGINEERING = "Engineering"
    SALES = "Sales"
    SUPPORT = "Customer Support"

print("Select a department:")
for index, dept in enumerate(Department, start=1):
    print(f"{index}. {dept.value}")

choice = int(input("Enter your choice (1-3): "))
selected = list(Department)[choice - 1]
print(f"You selected: {selected.value}")
```

In this example, the loop prints all departments as numbered menu options. Using enumerate() provides an automatic count, while converting the Enum to a list allows you to retrieve the user's selection by index. This keeps your interface consistent and flexible without hard-coded options.

Practical Uses

- Generating selection menus in console or GUI applications.
- Populating dropdown lists or combo boxes dynamically.
- Presenting configuration or setup options, such as difficulty levels or product categories.
- Validating user input by checking selections against defined Enum members.

In this example, each Employee object stores its Role as an Enum member. This ensures that only predefined roles can be assigned, reducing data errors and improving clarity.

4.9 Practice Exercises

1) Animal Hierarchy – Demonstrate overriding and polymorphism.
2) Shapes and Areas – Create subclasses of Shape to calculate areas.
3) Vehicle Hierarchy – Use super() to link Vehicle, Car, and Truck.
4) Multiple Inheritance – Implement a mixin with additional behavior.
5) Payment Types – Explore isinstance() and polymorphism together.

Key Takeaways

- Inheritance models 'is-a' relationships for code reuse and specialization.
- Use super() to extend rather than replace parent behavior.
- Polymorphism lets different objects respond to the same call in their own way.
- isinstance() and issubclass() can check relationships but should be used sparingly.
- Use multiple inheritance carefully—prefer composition or mixins when appropriate.
- Enums provide named constants that are grouped logically and safely.
- They demonstrate inheritance by extending the built-in Enum class.
- IntEnum and StrEnum combine Enum behavior with int or str operations.
- Enums improve readability, consistency, and reduce magic values in code.